



Universidade do Minho
Departamento de Informática

Aprendizagem Profunda

Trabalho Prático I

Grupo 11

João Coelho

PG55954

João Faria

PG55953

Jorge Teixeira

PG55965

Rafael Alves

PG55999

2025-04-03

Índice

1. Introdução	4
2. Datasets	4
3. Modelos em Numpy - Fase 1	4
3.1. Pré Processamento	4
3.2. Treino dos modelos	4
3.2.1. Logistic Regression	5
3.2.2. Deep Neural Network	5
3.2.3. Recurrent Neural Network	5
3.3. Comparação dos Resultados	5
4. Modelos em Tensorflow - Fase 2	6
4.1. Pré Processamento	6
4.2. Treino dos modelos	6
4.2.1. Recurrent Neural Network	6
4.2.2. Transformer	7
4.2.3. Deep Neural Network	7
4.2.4. Ensemble	7
4.3. Comparação dos Resultados	7
5. Modelos em Tensorflow - Fase 3	8
5.1. Mudanças na estrutura	8
5.2. Treino dos Modelos	8
5.2.1. Deep Neural Network	8
5.2.2. Recurrent Neural Network - LSTM	9
5.2.3. Recurrent Neural Network - GRU	9
5.2.4. Convolutional Neural Network	9
5.2.5. Transformer	9
5.2.6. Ensemble	9
5.3. Comparação dos Resultados	9
6. Conclusão	10

Lista de Figuras

Figura 1 Resultados Obtidos na Fase 1	6
Figura 2 Resultados Obtidos na Fase 2	8
Figura 3 Resultados Obtidos na Fase 3	10

1. Introdução

Este relatório apresenta o desenvolvimento do primeiro projeto da unidade curricular de Aprendizagem Profunda. O objetivo principal deste projeto é a construção de modelos de *Deep Learning* capazes de distinguir entre textos gerados por Inteligência Artificial e textos escritos por seres humanos.

Para alcançar este objetivo, os modelos serão treinados utilizando *datasets* em língua inglesa, sendo posteriormente avaliados com *datasets* com conteúdo relacionado aos tópicos da área da química, física, biologia e biomedicina.

Ao longo deste relatório, o processo de construção do *dataset* e as decisões tomadas serão abordadas fase a fase, detalhando os procedimentos adotados e os desafios enfrentados no desenvolvimento dos modelos.

2. Datasets

Ao longo do desenvolvimento deste projeto o(s) *dataset(s)* utilizado foi variando bastante. Numa primeira fase, utilizamos os *datasets* existentes e disponibilizados pela equipa docente no enunciado, mais especificamente os disponíveis na *HuggingFace*. Estes, apesar de se aparentarem positivos para o treino do modelo, visto que os mesmos seriam avaliados pela equipa docente usando um *dataset* temático, passaram para segundo plano, focando-nos na criação de um *dataset* usando informação retirada da *Wikipedia* como fonte de dados humanos e a API do *Gemini* para as entradas referentes a Inteligência Artificial. Assim sendo, conseguimos recolher um *dataset* com cerca de 5000 entradas sobre diversos tópicos relacionados com física, biologia e mecânica, à semelhança dos *datasets* de teste disponibilizados pela equipa docente para testarmos os nossos modelos.

3. Modelos em Numpy - Fase 1

Na área da Inteligência Artificial, o *Numpy* serve para otimizar operações numéricas, tornando mais rápidos e eficientes os cálculos numéricos. Estas potencialidades serviram como base para os modelos de *Logistic Regression* (LR), *Deep Neural Network* (DNN) e *Recurrent Neural Network* (RNN).

3.1. Pré Processamento

Antes de treinar os modelos, é necessário um pré processamento do *dataset*. Para esta fase, como referido anteriormente, utilizamos um dos *datasets* da *HuggingFace*, mais especificamente o *dmitva/human_ai_generated_text*, lendo os dados em lotes (para o efeito, usamos 25 mil). A informação no *dataset* é limpo, substituindo maiúsculas, pontos e as chamadas “*stopwords*”, obtidas da biblioteca *nlk*. Depois, do *dataset* de treino, são obtidos os valores da coluna objetivo e transformados para valores numéricos. Após isso, o *dataset* é equilibrado entre as duas classes e vetorizado usando o *TfidfVectorizer* do *sklearn*, mantendo apenas as 1000 palavras mais frequentes. É ainda treinado um modelo *Word2Vec* usando a biblioteca *gensim* que aprende representações vetoriais com um vetor de tamanho 300, janela de 5 palavras e mínimo de 2 ocorrências por palavra. Por fim, é criada a matriz de *embedding* onde se associa uma palavra a um vetor, criando sequências numéricas a partir dos textos, substituindo as palavras por índices na matriz de *embeddings*. O *dataset* de validação também é pré-processado, da mesma forma. Por fim, os resultados são armazenados (modelo, *TfidfVectorizer*, matriz de *embeddings*, dados de treino e de validação e mapeamento de palavras para índices).

3.2. Treino dos modelos

De seguida, passamos para o treino dos 3 modelos desenvolvidos ao longo desta primeira fase.

3.2.1. Logistic Regression

Foi desenvolvido um modelo de Regressão Logística para realizar a classificação dos dados. Após carregar os mesmos, o modelo de Regressão Logística é definido com uma taxa de aprendizagem de 0.01 e configurado para treinar por 300 *epochs*. O objetivo do mesmo é encontrar a melhor linha de separação entre as classes.

3.2.2. Deep Neural Network

O melhor modelo de DNN para a fase utilizando apenas *Numpy*, além da camada de entrada ser do tamanho do *dataset* de treino, a primeira camada oculta é constituída por uma camada totalmente conectada com 32 neurónios e regularização *L2* (peso de 0.05), com normalização das ativações (para estabilizar o treino), função de ativação *ReLU* e, de forma procurar reduzir o *overfit*, foi adicionado um *dropout* de 50. A segunda camada oculta é igual à primeira, mas com uma camada de apenas 16 neurónios. Já a camada de saída contém apenas um neurónio, de forma a ter uma saída binária, com uma função de ativação sigmoide.

Relativamente a otimização e estratégias de treino, foi utilizada a *loss function* de *FocalLoss*, focando-se na métrica *accuracy*, com um *learning rate* de 0,0001 e *momentum* de 0,9. O *batch size* é 128 com 100 *epochs* e *early stopping* com paciência de 5 *epochs*. O *scheduler* de *learning rate* tem um decaimento de 0,2 e existe *Mixup Data Augmentation* com *mixup_alpha*=0.2.

3.2.3. Recurrent Neural Network

A RNN tem uma entrada que consiste em sequências previamente processadas (*X_train_seq* e *X_val_seq*), juntamente com a coluna objetivo (*y_train* e *y_val*). Além disso, é utilizada uma matriz de *embeddings* (*embedding_matrix*), que transforma palavras em representações numéricas densas, facilitando a captura de relações semânticas.

A arquitetura da RNN inclui uma camada recorrente com 64 unidades e *dropout* de 50%, uma técnica que ajuda a reduzir o *overfitting* ao desativar aleatoriamente algumas unidades durante o treino. A otimização é feita com o algoritmo *Adam*, um método avançado de descida do gradiente com taxas de aprendizagem adaptativas (*lr*=0.0005). O treino ocorre ao longo de 15 *epochs*, com um tamanho de *batch* de 16, permitindo que o modelo aprenda padrões na sequência de dados.

3.3. Comparação dos Resultados

Após o desenvolvimento dos modelos, foi utilizado o conjunto de *datasets*, juntos num único (130 entradas), disponibilizados pela equipa docente de forma a procurar validar os modelos relativamente à sua *accuracy*. Estes foram os resultados obtidos para o mesmo:

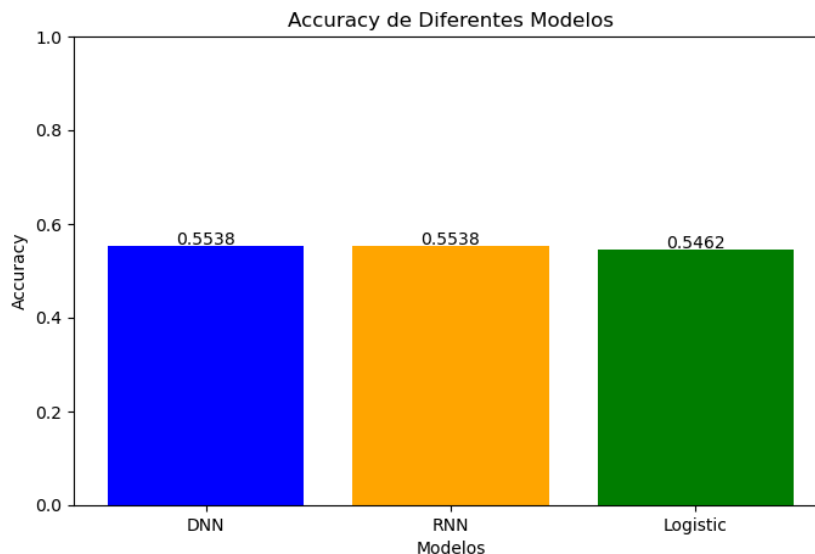


Figura 1: Resultados Obtidos na Fase 1

4. Modelos em Tensorflow - Fase 2

Passando agora para a segunda fase, desenvolvemos diversos modelos mas agora podendo recorrer a diversas bibliotecas de *machine learning* de python, como por exemplo o *TensorFlow*.

4.1. Pré Processamento

Comparativamente ao pré processamento da fase anterior, neste foi utilizado o *dataset* próprio gerado que foi mencionado previamente, dividido em treino e validação, além do uso do *Tokenizer* e *pad_sequences* do *TensorFlow*, ao invés do *TfidfVectorizer*, foi feita a extração de *features*, como o comprimento médio das frases, diversidade lexical, frequência de pontuação e frequência de pronomes de primeira pessoa). Nesta fase, é treinado um modelo *Word2Vec* e são armazenados os *embeddings* como uma matriz customizada (*create_custom_embeddings*), enquanto o código anterior treinava *Word2Vec* apenas para análise sem essa integração direta com *TensorFlow*. Além disso, os metadados são guardados, a par dos dados processados.

4.2. Treino dos modelos

Relativamente ao treino de modelos desta fase, ele é iniciado com o carregamento das sequências de textos (vetorizadas para modelos de *deep learning*), rótulos (*labels* de classificação), matriz de *embeddings* (pré-treinada, usada para representar palavras num espaço vetorial), metadados, *tokenizer*, textos originais e *features* adicionais.

4.2.1. Recurrent Neural Network

A nível de modelos baseados em RNNs foram implementados *LSTM* e *GRU*. O *LSTM* utiliza uma camada de *embedding* inicializada com uma matriz de *embeddings* pré-treinada e configurada com um tamanho máximo de sequência. Este consiste em duas camadas *bidirecionais* com 128 e 64 unidades, respetivamente, com dropout (também recurrent) de 30%. Este modelo inclui uma camada densa de 32 neurônios com ativação *ReLU* e regularização L2 (0.001), seguida de uma camada de saída *sigmoid* que, como é claro, tem apenas um único neurônio.

Já o *GRU* tem uma estrutura semelhante, mas com uma camada *Conv1D* com 128 filtros antes das camadas *bidirecionais*. A camada densa com neurônios tem a mesma função de ativação e regularização. Ambos utilizam otimizador *Adam* com *learning rate* de 0.001 e função de perda *binary_crossentropy*.

4.2.2. Transformer

O *Transformer* recebe como entrada uma camada de *Embedding* inicializada com uma matriz pré-treinada. Além de múltiplos blocos de *self-attention* onde cada um tem cabeças de atenção de 64 (8, no total) contém uma camada *feed-forward* de 256 unidades. Usando extração de características com *GlobalAveragePooling1D*, o modelo passa por camadas densas com ativação *ReLU* e regularização L2. Apesar de seguir a mesma configuração dos outros modelos, tem um *fine tuning* com *learning rate* de 0.0005.

4.2.3. Deep Neural Network

A *DNN*, totalmente conectada, trabalha sobre *features* tabulares extraídas dos textos. Para além disso, a sua rede tem camadas densas de 128, 64 e 32 neurônios, ativação *LeakyReLU*, *Batch Normalization* e *Dropout* (50%) para regularização. Além disso, utiliza *L2 regularization* (0.001) para evitar *overfitting* e termina com uma camada *sigmoid* para saída probabilística. Compilado com *Adam* (lr=0.001) e *binary_crossentropy*.

4.2.4. Ensemble

Por fim, a arquitetura de *ensemble* combina os modelos anteriormente abordados, já treinados para produzir uma *predict* final mais robusta. Esse modelo recebe como entrada as saídas de cada um dos modelos individuais e processa-as recorrendo a camadas densas com ativação *ReLU*, regularização L2 e *dropout*, refinando a decisão final.

Além disso, é chamada a função *evaluate_model()* que calcula métricas de desempenho para um modelo treinado como a matriz de confusão, curva ROC e AUC (mede a capacidade do modelo de distinguir entre as classes) e a *accuracy*.

O modelo *GRU* e o *Ensemble* foram utilizados para as *predicts* utilizadas na submissão para avaliação da equipa docente, visto serem os modelos que deram melhores resultados no treino.

4.3. Comparação dos Resultados

Relativamente aos modelos desenvolvidos para esta fase, os resultados não foram os melhores, como podemos ver no gráfico seguinte:

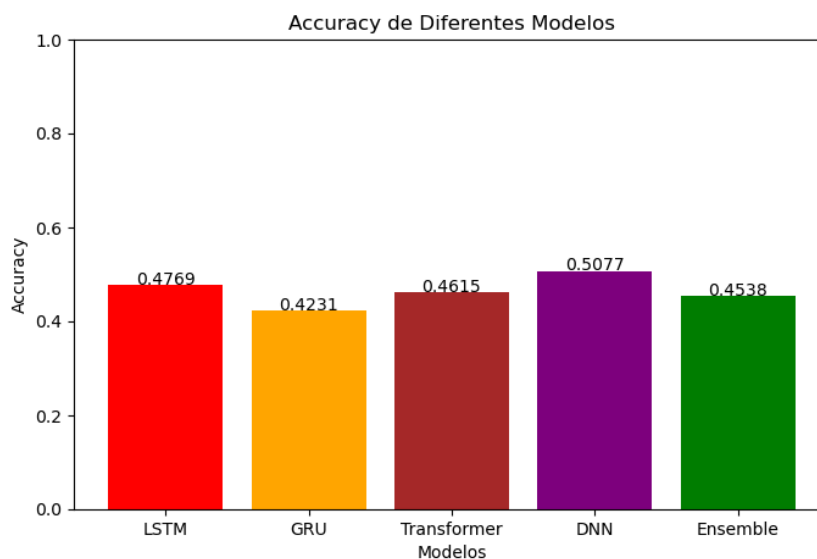


Figura 2: Resultados Obtidos na Fase 2

O baixo desempenho dos nossos modelos nesta fase foi o que nos incentivou a procurar uma melhor organização estrutural do trabalho, bem como encontrar formas de simplificar o processo de pré-processamento dos dados e treinamento dos modelos, que, na nossa opinião, estavam excessivamente complicados e poderiam ser algumas das coisas do fraco desempenho.

5. Modelos em Tensorflow - Fase 3

Finalmente, chegamos à terceira e última fase do trabalho prático. Nesta, o objetivo era apenas aplicar possíveis melhorias aos modelos *tensorflow* apresentados anteriormente na fase 2.

5.1. Mudanças na estrutura

Assim sendo, para esta fase foram substituídos os ficheiros separados de *Python* por *Jupyter Notebooks*, cada um deles dedicado para um modelo, onde todo o pré-processamento e treino são realizados diretamente. Também o pré-processamento foi alterado, abandonando o *Word2Vec*, passando a ser utilizado *embedding* treinável com *GlorotUniform*. Além disso, a utilização de *Keras Tuner*, algo que só foi descoberto pelos elementos do grupo nesta fase, permitiu ajustes mais eficientes dos hiperparâmetros dos modelos, levando a resultados significativamente melhores quando comparados com os das fases anteriores.

5.2. Treino dos Modelos

De seguida, como é esperado, devido à nova estrutura dos modelos, bem como as novas técnicas de *fine tuning* aplicadas, o treino dos modelos sofreu diversas alterações.

5.2.1. Deep Neural Network

O pré-processamento deste modelo passou a consistir em normalização, remoção de maiúsculas, *URLs* e *emails*. A nível de arquitetura, foram utilizadas 4 camadas ocultas com 216, 88, 216 e 120 neurônios, respetivamente. A ativação foi feita com *ReLU* e normalização e os *dropouts* passaram a ser de 10% nas duas primeiras camadas, 40% na terceira e 30% na quarta. A camada de saída manteve-se de 1 neurônio com ativação sigmoide. O treino foi feito com o otimizador *Adam*, *binary crossentropy* e *early stopping* de 2 *epochs*. O modelo final tem, ainda *Early Stopping* combinado com *ReduceLROnPlateau*.

5.2.2. Recurrent Neural Network - LSTM

O pré-processamento deste modelo consiste num *tokenizer* e tratamento de dados idêntico ao aplicado na DNN. As camadas começam com uma de *embedding* de dimensão 64 e *Spatial Dropout* de 20%, seguida de uma camada LSTM de 40 unidades, *dropout* de 50% e *recurrent dropout* de 20%, seguida de outra camada de 64 unidades, *dropout* de 20% e *recurrent dropout* de 40%. A última camada é densa e de apenas uma unidade (sigmoide). A otimização utilizou Adam e *binary crossentropy*. O treino foi feito com Early Stopping por 5 *epochs* e *ReduceLROnPlateau*. Já a configuração teve, no máximo, 25 *epochs* e *batch size* de 32.

5.2.3. Recurrent Neural Network - GRU

As camadas deste modelo consistem, inicialmente, por uma de *embedding* de dimensão 64, usando *GlorotUniform* e com *Spatial Dropout* de 30%, seguida de uma camada *Conv1D* com *ReLU*. As camadas GRU são duas, uma de 56 unidades, *dropout* de 20% e *recurrent dropout* de 30%, sendo a segunda de 8, 10% de *dropout* e 20% de *recurrent dropout*. Por fim, a camada densa de ativação sigmoide. A otimização utilizou Adam e *binary crossentropy* e treinado com *early stopping* de 5 *epochs* e *ReduceLROnPlateau*, configurado com 25 *epochs*, no máximo, e *batch size* de 32.

5.2.4. Convolutional Neural Network

A arquitetura é composta por uma camada de *embedding*, seguida por múltiplas camadas convolucionais (*Conv1D*) com tamanhos de filtros variados (2, 3, 4 e 5) e operações de *Global Max Pooling*. As saídas dessas camadas são concatenadas e passam por camadas densas (*Dense*) com ativação *ReLU* e *dropout*. A camada final é densa, com uma única unidade e ativação sigmoide para classificação binária. O treino foi realizado com o otimizador Adam e a função de perda *binary crossentropy*, utilizando *Early Stopping* (paciente por 5 *epochs*) e *ReduceLROnPlateau* para ajuste dinâmico da taxa de aprendizagem.

5.2.5. Transformer

O pré-processamento deste modelo é exatamente igual ao aplicado aos modelos anteriores. As camadas começam com uma de *embedding* de dimensão 128, somada a uma camada de *positional encoding*. Seguem-se 4 blocos *Transformer*, cada um composto por uma camada de *Multi-Head Attention* com 4 cabeças e dimensão de 32, normalização por camadas (*Layer Normalization*) e uma rede *feed-forward* com dimensão 256. Cada bloco inclui *dropout* de 10%. Após os blocos *Transformer*, os dados passam por uma camada de *Global Average Pooling* e duas camadas densas com 128 e 64 unidades, respetivamente, ambas com ativação *ReLU* e *dropout* de 10%. A última camada é densa, com uma unidade e ativação sigmoide, para conseguirmos obter os resultados com valor entre 0-1. A otimização utilizou Adam com uma taxa de aprendizagem inicial de (10^{-4}) e a função de perda *binary crossentropy*. O treino foi realizado com *Early Stopping* (paciente por 5 *epochs*) e *ReduceLROnPlateau*, com um máximo de 30 *epochs* e *batch size* de 32.

5.2.6. Ensemble

Finalmente, chegamos ao *ensemble* que, por sua vez, tem como objetivo juntar os modelos anteriores a um *Transformer*. A primeira camada densa tem 48 neurônios com *dropout* de 50%, a segunda camada densa de 32 neurônios e *dropout* de 30% e uma taxa de aprendizagem de aproximadamente 0,0018 e otimizador Adam. O *tuner* está configurado com 2 *epochs* iniciais e 2 *brackets*.

5.3. Comparação dos Resultados

Finalmente, à semelhança das duas fases anteriores, desenvolvemos um gráfico que demonstra os valores da *accuracy* obtidos pelos diferentes modelos no *dataset* de teste disponibilizado pela equipa docente.

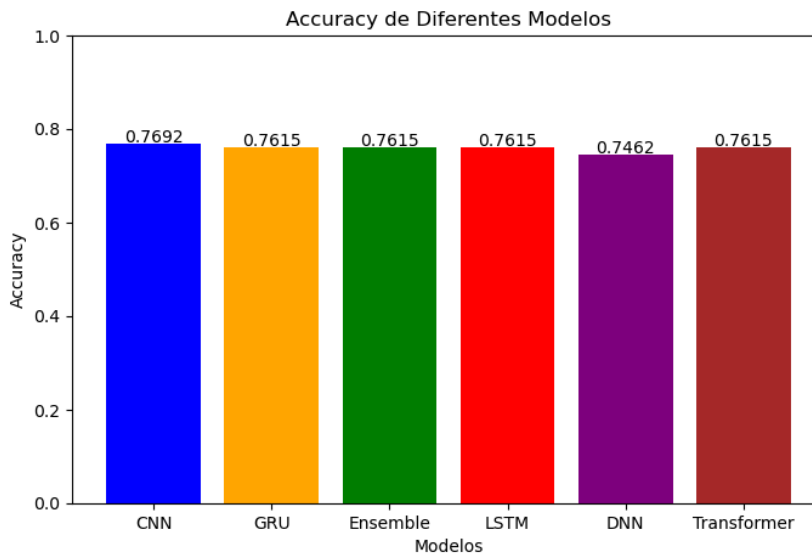


Figura 3: Resultados Obtidos na Fase 3

Desta forma, podemos concluir que, na generalidade, os modelos obtiveram resultados satisfatórios dentro do contexto do problema. Para além disso, é possível verificar uma evolução, e respetiva melhoria, do desempenho dos modelos ao longo das diferentes fases do trabalho. Isto demonstra que o grupo, recorrendo ao *feedback* fornecido em cada etapa do trabalho, conseguiu aprimorar as diferentes fases de desenvolvimento do projeto e, graças a tal, foi possível obter uma melhoria gradual dos modelos.

6. Conclusão

Este projeto foi bastante desafiante, levando-nos a uma compreensão mais aprofundada dos modelos de inteligência artificial, mais propriamente da área do *Deep Learning*, solidificando conceitos apresentados durante as aulas através da prática e do desenvolvimento.

A nível de conceitos técnicos, foi de especial relevância a correta utilização e tratamento de *datasets*, alinhada com a inserção de dados de maior qualidade dentro dos tópicos proposto, revelando o papel central desta parte na inteligência artificial. Além disso, a procura dos melhores hiperparâmetros revelou-se fulcral na busca de melhorias nos modelos e a exploração dos mesmos acrescentaram informação valiosa para desenvolvimentos futuros nesta área, a par da utilização de bibliotecas como o *TensorFlow*, de especial importância no contexto da indústria tecnológica dentro deste setor.